Lab 6 – Inverse Kinematics

1 Introduction

In this lab the inverse kinematics of a SCARA manipulator will be implemented in C++ and simulated.

2 Experiment

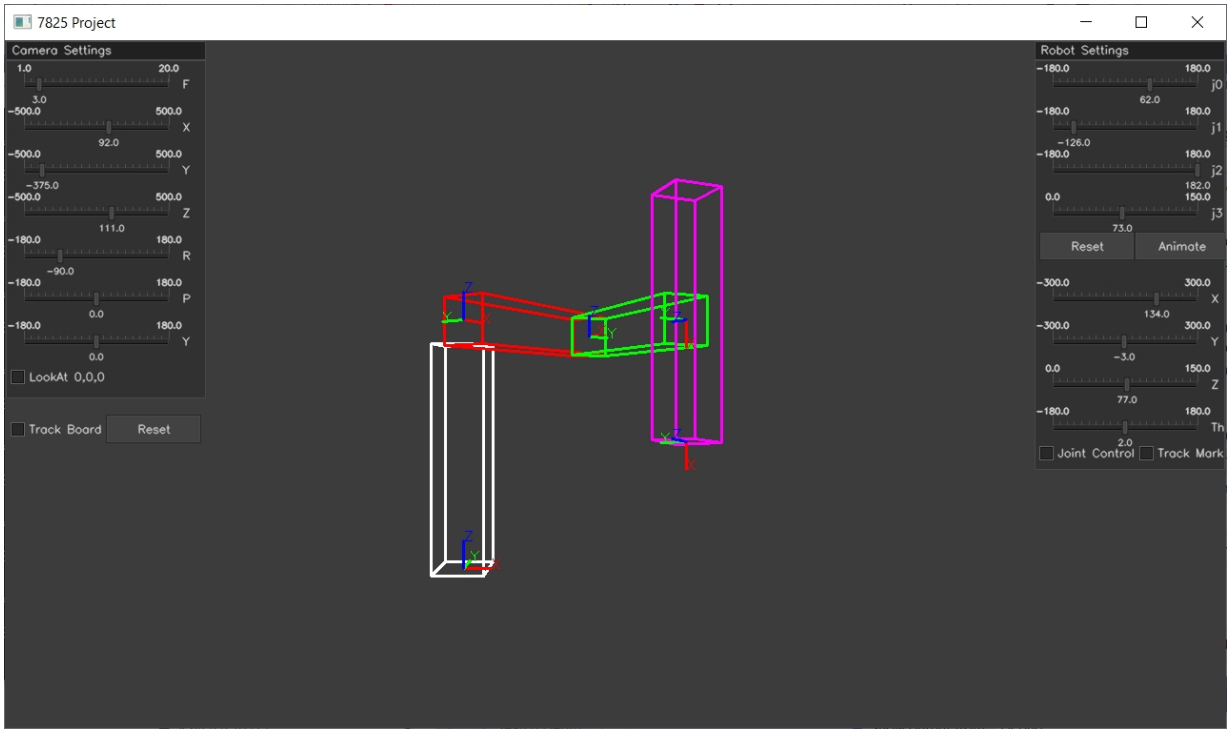
2.1 SCARA Inverse Kinematics Part A

A) Given a target end effector pose, determine the closed form inverse kinematic equations for the SCARA robot (solve by hand or use symbolic variables in MATLAB). The solution to the 2-link planar manipulator provides the inverse kinematics for the X and Y position. The prismatic joint can be determined based on the target Z position. The last joint angle can be determined by using the first 2 joint angles determined above to compute the end effector orientation (can use ***fkin***), and then subtracting the end effector orientation from the target end effector orientation.

B) Implement the inverse kinematics into an ***ikine*** method for the CRobot class with an input of target pose (X, Y, Z, θ) and output of the joint angles (j0, j1, j2, j3).

C) Add a checkbox to toggle to switch between joint control to end effector control using the robot GUI sliders. For a given target input, compute the required joint angles and display on the sliders as well as graphically with the pose of the robot arm.

D) Add an inverse kinematic animation to the animate button, which previously moved the joints in joint control mode. Have the end effector move along a linear path through the workspace.



2.2 SCARA Inverse Kinematics Part B

A) Draw the SCARA robot at the origin of the ArUco board, the same as in lab 5. Show that the joint animation still works.

B) Add a checkbox to the CVUI menu to enable tracking of an additional ArUco marker (ID = 50). Determine the 3D pose (X,Y,Z,R,P,Y) of the marker using the `estimatePoseSingleMarkers` function. NOTE this function returns the rotation matrix in a different format, to convert to our standard format use the `Rodrigues` function. Draw a 3D box around the ArUco marker and display.

C) Track the marker with the robot by linking the 3D marker pose (X,Y,Z,θ) to the CVUI pose display and use this data as the input to the inverse kinematics function. The robot should now be locked to the tracking marker.

